COMP 2233: DATA STRUCTURES & ALGORITHMS

WEEK 4 LECTURE NOTES

Single-Dimensional Arrays, Dynamic Vectors, and Multidimensional Array Architectures

Instructor: Assoc. Prof. Dr. Ömer ÇETİN | Academic Term 2026

Table of Contents

1. Executive Summary & Module Overview

Arrays and vectors represent the foundational contiguous memory structures in software engineering. Module 4 explores the physical RAM layout, memory address calculation formulas, operational complexity, and programming techniques for single-dimensional static arrays, C++ STL dynamic vectors, and multidimensional matrix structures. Understanding these structures provides the groundwork for hardware-level optimizations, CPU cache prefetching, and advanced non-linear data structures.

> **KEY INSIGHT: Memory Contiguity Drives CPU Performance**
> Physical RAM contiguity enables O(1) random element indexing because memory addresses are calculated using deterministic arithmetic rather than pointer traversal. Furthermore, contiguous memory blocks maximize CPU Spatial Cache Hit ratios, dramatically outperforming pointer-linked structures during iteration.

2. Single-Dimensional Arrays in C

A single-dimensional array is a fixed-size, contiguous sequence of homogeneous data elements stored in consecutive memory slots. In C, arrays provide zero-based indexing, allowing direct element access in constant time O(1).

2.1 Memory Architecture & Address Calculation

Because array elements are stored in adjacent RAM addresses, the physical memory address of any element at index *i* is derived mathematically using the base address of the first element **A[0]** and the byte size of the element data type:

```
Address(A[i]) = BaseAddress + (i * sizeof(DataType))
```

Listing: Single-Dimensional Array Random Access Address Formula

For instance, if an integer array **int arr[4]** starts at Base Address **0x1000** and each integer occupies 4 bytes, the physical layout is mapped as follows:

Index (i)	Element Value	Memory Address Range	Byte Offset Calculation
0	10	0x1000 - 0x1003	0x1000 + (0 * 4) = 0x1000
1	20	0x1004 - 0x1007	0x1000 + (1 * 4) = 0x1004
2	30	0x1008 - 0x100B	0x1000 + (2 * 4) = 0x1008
3	40	0x100C - 0x100F	0x1000 + (3 * 4) = 0x100C

2.2 Fundamental Array Operations in C

Static arrays support 8 core operations. While direct index access and updating execute in **O(1)** time, operations involving mid-array insertions or deletions require element shifting and run in **O(N)** time.

```
#include <stdio.h>
#define CAPACITY 100

// 1. Traversing O(N)
void traverseArray(const int arr[], int size) {
    printf("Array: [ ");
    for (int i = 0; i < size; i++) printf("%d ", arr[i]);
    printf("]\n");
}

// 2. Insertion O(N) - Right Shift
int insertAt(int arr[], int size, int capacity, int pos, int val) {
    if (size >= capacity || pos < 0 || pos > size) return size;
    for (int i = size - 1; i >= pos; i--) arr[i + 1] = arr[i];
    arr[pos] = val;
    return size + 1;
}

// 3. Deletion O(N) - Left Shift
int deleteAt(int arr[], int size, int pos) {
    if (pos < 0 || pos >= size) return size;
    for (int i = pos; i < size - 1; i++) arr[i] = arr[i + 1];
    return size - 1;
}

// 4. Searching O(N) - Linear Search
```

```c
int linearSearch(const int arr[], int size, int key) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == key) return i;
    }
    return -1;
}
```

Listing: Core Array Operations Implementation in C

2.3 Struct Encapsulation for Self-Aware Array ADT

Raw C arrays decay into pointers when passed to functions, losing their size information. Encapsulating an array alongside its active **size** and maximum **capacity** inside a **struct** builds a self-contained Abstract Data Type (ADT).

```c
typedef struct {
    int data[100];
    int size;
    int capacity;
} ArrayADT;

void printADT(const ArrayADT *arr) {
    printf("ADT Size: %d/%d | Elements: ", arr->size, arr->capacity);
    for (int i = 0; i < arr->size; i++) {
        printf("%d ", arr->data[i]);
    }
    printf("\n");
}
```

Listing: Encapsulated Array ADT Structure in C

3. Dynamic Vectors in C++ (STL std::vector)

The C++ Standard Template Library (STL) **std::vector** provides a dynamic array container that automatically expands and contracts its memory capacity at runtime while preserving contiguous RAM storage and O(1) random access.

3.1 Size vs. Capacity & Memory Doubling Mechanics

A vector distinguishes between its active element count (**size**) and its total allocated memory buffer (**capacity**). When a **push_back()** call occurs and **size == capacity**, the vector allocates a new memory block of **2 * Capacity**, copies existing elements over, and frees the old block. This geometric growth strategy yields an **amortized O(1)** constant time for insertion.

```c
#include <iostream>
#include <vector>

int main() {
    std::vector<int> vec;
```

```
    for (int i = 1; i <= 5; i++) {
        vec.push_back(i * 10);
        std::cout << "Val: " << i*10
                  << " | Size: " << vec.size()
                  << " | Cap: " << vec.capacity() << "\n";
    }

    // Bounds-checked access
    try {
        std::cout << "Element at 2: " << vec.at(2) << "\n";
        std::cout << vec.at(10); // Throws std::out_of_range
    } catch (const std::out_of_range& e) {
        std::cerr << "Exception caught: " << e.what() << "\n";
    }
    return 0;
}
```

Listing: Dynamic Vector Operations & Bounds Safety in C++

3.2 Comparative Analysis: Static C Array vs. Dynamic C++ Vector

Feature / Dimension	Static C Array (char[] / int[])	Dynamic C++ Vector (std::vector<T>)
Memory Allocation	Fixed at compile-time (Stack / Heap)	Dynamic runtime expansion (Heap)
Size Management	Manual tracking via external variable	Automated via .size() and .capacity()
Random Access Speed	O(1) direct offset arithmetic	O(1) direct offset arithmetic
Bounds Safety	Unchecked (Risks Segmentation Fault)	Safe via .at(index) exception handling
Memory Cleanup	Manual free() for dynamic arrays	Automated via RAII class destructors

4. Multidimensional Arrays & Matrix Operations

A multidimensional array organizes elements in grid formats (2D matrices, 3D spatial volumes). Although conceptualized as tables with rows and columns, computer RAM is linear (1D). Therefore, multidimensional arrays are flattened into 1D memory spaces using Row-Major or Column-Major order.

4.1 Row-Major Order & Address Formula

In C/C++, matrices are stored in **Row-Major Order**, meaning row 0 is written entirely into contiguous memory, followed immediately by row 1, row 2, and so on. To locate element **A[i][j]** in a matrix with **COLS** columns, the offset formula is:

```
Address(A[i][j]) = BaseAddress + ((i * COLS) + j) * sizeof(DataType)
```

Listing: 2D Row-Major Order Memory Address Formula

> **KEY INSIGHT: CPU Cache Locality & Loop Ordering**
> Because matrices are stored row-by-row, traversing a matrix row-first (outer loop i, inner loop j) accesses consecutive memory addresses, producing a Spatial Cache Hit. Traversing column-first (outer loop j, inner loop i) jumps by COLS * sizeof(DataType) bytes, causing frequent Cache Misses and severe CPU slowdowns.

4.2 Matrix Operations Implementation in C++

Matrix multiplication requires matching inner dimensions (A cols == B rows) and executes in **O(N^3)** time using triple nested loops:

```
#include <iostream>
#include <vector>

// Matrix Multiplication C++ Implementation
std::vector<std::vector<int>> multiplyMatrices(
    const std::vector<std::vector<int>>& A,
    const std::vector<std::vector<int>>& B,
    int R1, int C1, int C2)
{
    std::vector<std::vector<int>> C(R1, std::vector<int>(C2, 0));

    // O(N^3) Triple Nested Loop
    for (int i = 0; i < R1; i++) {
        for (int j = 0; j < C2; j++) {
            for (int k = 0; k < C1; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
    return C;
}
```

Listing: 2D Matrix Multiplication Implementation in C++

5. Review Questions & Practical Lab Exercises

1. Calculate the physical RAM address of element A[3][2] in a 4x4 integer matrix starting at Base Address 0x2000 in Row-Major Order (assume sizeof(int) = 4 bytes).

2. Explain why std::vector::push_back() operates in amortized O(1) time despite triggering capacity doubling reallocations.

3. Compare the spatial cache performance of traversing a 2D matrix row-wise versus column-wise in C++.

4. Write a C function that reverses a single-dimensional array in-place using a two-pointer technique in O(N) time.